SciToolAgent-Evo: An Ontology-Aware Self-Evolving Agent for Open-World Scientific Tool Acquisition

Yuqi Tang[1,2], **Chenyi Zhou**[1], **Libin Wang**[3], **Keyan Ding**[2], **Qiang Zhang**[2,3*], **Huajun Chen**[1,2*]

[1]College of Computer Science and Technology, Zhejiang University
[2]ZJU-Hangzhou Global Scientific and Technological Innovation Center, Zhejiang University
[3]ZJU-UIUC Institute, Zhejiang University
yuqi.22@intl.zju.edu.cn

arXiv:2607.28692v1 [cs.AI] 30 Jul 2026

Abstract

Large language model (LLM) agents have been increasingly adopted in scientific research for organizing and invoking specialized computational tools. However, their reliance on predefined tool spaces with static semantics limits their applicability to open-world scientific workflows, where tool requirements, capabilities, and boundaries evolve dynamically. To this end, we propose SciToolAgent-Evo, an ontology-aware self-evolving agent for open-world scientific tool acquisition. Driven by an evolving memory of skills, experiences, and an ontologized tool graph, it distills generalizable knowledge from contrastive trajectories during accumulation, whereas during inference, it formulates active requests and utilizes a LinUCB-based bandit gate to dynamically balance exploration and exploitation. Once a novel tool is acquired, its scientific ontology is completed online for seamless integration into the known graph. Moreover, we introduce OpenSciToolBench, a benchmark containing 900 realistic tasks across four difficulty levels. Extensive evaluations show that SciToolAgent-Evo achieves state-of-the-art performance, validating its robustness and generalization.

1 Introduction

Large language model (LLM) agents have demonstrated remarkable intelligence in complex interactive tasks (Min et al. 2023; Vemprala et al. 2024; Liu et al. 2024; Tang et al. 2025), driving their growing adoption in scientific research to organize and invoke specialized computational tools (Wang et al. 2024; Wu et al. 2024; Chaudhari et al. 2026; Ghareeb et al. 2026). However, real scientific workflows rarely begin with a complete and perfectly specified toolbox (Mekala et al. 2024). For example, a scientific agent may be asked to identify a molecule satisfying multiple constraints, but the required workflow may involve a descriptor calculator or conversion tool absent from its known tool space. In such cases, closed-world agents are confined to existing tools and often fail when the required capabilities are missing.

This observation suggests a shift from closed-world tool selection to open-world scientific tool acquisition. Advanced scientific tool-use frameworks, such as ChemCrow (M. Bran et al. 2024), GeneGPT (Jin et al. 2024), ChatMOF (Kang and Kim 2024), and SciToolAgent (Ding et al. 2025) can retrieve, select, and invoke tools from large-scale libraries (Tang et al. 2023). However, their reliance on predefined tool spaces with static semantics hinders capability-gap recognition, leading them to passively over-exploit the known-tool space even when it is insufficient. Moreover, without a self-evolving consolidation mechanism, they struggle to reliably ground and reuse newly acquired tool knowledge in future tasks.

Meanwhile, existing scientific tool-use benchmarks (Ma et al. 2024a; Chen et al. 2025; Ding et al. 2025) typically involve short tool chains, with task descriptions that contain explicit references to tool names. Such designs make them insufficient for evaluating the core capabilities required for open-world scientific tool acquisition: recognizing capability gaps, balancing known-tool exploitation with unknown-tool exploration, and continually consolidating tool knowledge.

To address these challenges, we propose SciToolAgent-Evo, an ontology-aware self-evolving agent for open-world scientific tool acquisition. Specifically, SciToolAgent-Evo maintains an evolving memory consisting of a skill library, an experience pool, and an ontologized tool graph for continual knowledge consolidation and reuse. During accumulation, the agent distills compact tool-use knowledge from contrastive trajectories. At inference time, it formulates active requests and employs a LinUCB-based bandit gate to balance exploration and exploitation. Once an unknown tool is acquired, its ontology is completed online and integrated into the known graph. Additionally, we construct OpenSciToolBench, a hierarchical evaluation benchmark for open-world scientific tool use. Extensive experiments across scientific tool-use benchmarks demonstrate that SciToolAgent-Evo exhibits strong robustness and generalization. Our main contributions are as follows:

- We propose **SciToolAgent-Evo**, an ontology-aware self-evolving agent for open-world scientific tool acquisition. It integrates an evolving memory, active tool requests, and an exploration–exploitation bandit gate to enable autonomous adaptation in dynamic scientific environments.
- We develop **OpenSciToolBench**, a hierarchical benchmark for open-world scientific tool use, containing 900 realistic tasks across four difficulty levels.
- We conduct extensive evaluations to demonstrate the superiority of SciToolAgent-Evo over competitive baselines. Detailed analyses further validate its core mechanisms, including evolving-memory consolidation, active

*Corresponding author.

tool requests, and the bandit gate.

2 Related Work

2.1 LLM Agents

LLM-based agents have shown strong abilities in complex tool-use tasks (Shen et al. 2023; Paranjape et al. 2023; Wang et al. 2025a; Qian et al. 2026). Frameworks such as ReAct (Yao et al. 2023), Reflexion (Shinn et al. 2023), Toolformer (Schick et al. 2023), and ToolLLM (Qin et al. 2024) improve tool planning and API matching through in-context learning, feedback-driven refinement, or targeted fine-tuning. Recent work further extends agents to scientific domains, with systems such as SciToolAgent (Ding et al. 2025) and SciAgent (Ma et al. 2024a) automating specialized computation and disciplinary reasoning. However, these methods largely act as passive selectors within predefined scientific tool spaces, leaving capability-gap recognition, unknown-tool exploration, and continual knowledge consolidation unresolved. We aim to fill this gap by introducing SciToolAgent-Evo, an ontology-aware self-evolving agent for open-world scientific tool acquisition.

2.2 Tool-Use Benchmarks

In recent years, the development of comprehensive tool-use benchmarks has gained increasing attention (Mekala et al. 2024; Lu et al. 2025; Chen et al. 2025). Early benchmarks, such as APIBank (Li et al. 2023) and RESTBench (Song et al. 2023), mainly evaluate static tool selection and invocation under explicit specifications. Subsequent benchmarks improve realism by introducing larger tool spaces and unseen-tool generalization; representative examples include Gorilla (Patil et al. 2024), ToolGen (Wang et al. 2025b), and AppWorld (Trivedi et al. 2024). In the scientific domain, SciToolEval (Ding et al. 2025) evaluates agents' multi-step API planning and execution capabilities, while SciToolBench (Ma et al. 2024a) assesses agents' ability to retrieve and reason with composable scientific functions. Despite their effectiveness, these benchmarks typically involve short tool chains and task descriptions with explicit tool-name cues, limiting their ability to evaluate the core capabilities required for open-world scientific tool acquisition. To this end, we construct OpenSciToolBench, a hierarchical benchmark for open-world scientific tool use.

3 OpenSciToolBench

This section presents the dataset construction process of OpenSciToolBench, including its benchmark taxonomy, question generation process, and quality control protocol.

3.1 Benchmark Taxonomy

Inspired by the Cognitive Rigor Matrix (Hess et al. 2009), we formulate four difficulty levels to characterize agents' capability boundaries in open-world scientific tool-use scenarios:

- **Level-1 (1–2 tools)**: Assessing explicit tool invocation under directly specified tool requirements.
- **Level-2 (2–4 tools)**: Evaluating property analysis through direct retrieval, computation, or comparison.

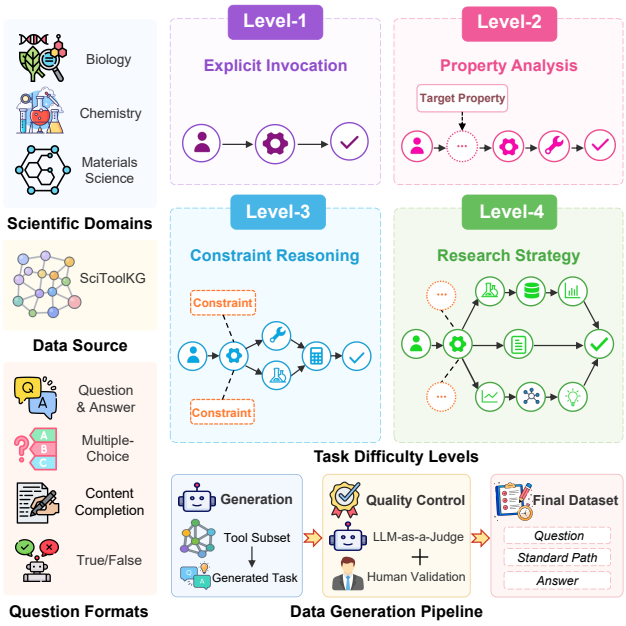


Figure 1: Overview of OpenSciToolBench. The benchmark contains 900 scientific tool-use questions constructed from SciToolKG, spanning three scientific domains, four question formats, and four task difficulty levels. The lower panel illustrates the data generation pipeline, including question generation and quality control to construct the final dataset.

- **Level-3 (3–6 tools)**: Evaluating constraint-aware reasoning that integrates multi-dimensional properties under multiple scientific constraints.
- **Level-4 (5–10 tools)**: Challenging open-ended research strategy formulation under realistic research goals.

These four levels form the foundation of OpenSciToolBench and provide a systematic framework for evaluating open-world scientific tool use.

3.2 Dataset Generation

Building on SciToolKG (Ding et al. 2025), which encompasses approximately 500 scientific tools across biology, chemistry, and materials science, we first randomly sample small tool subsets, ensuring that tools within each subset belong to the same domain and are connected through input-output compatibility. We organize their node information (e.g., descriptions, input/output types, sources) into task specifications. We then craft level-specific prompts and use GPT-5.1 (OpenAI 2025) to select reasonable tool paths from each subset and generate corresponding questions, ensuring that the generated tasks are scientifically meaningful and aligned with the required competencies.

Detailed prompts are provided in Appendix C. The resulting questions span four formats: question & answer, multiple choice, content completion, and true/false, enabling a comprehensive evaluation of open-world scientific tool use.

3.3 Quality Control

To ensure scientific rigor and tool-chain coherence, we adopt a two-stage verification process:

- *LLM-as-a-Judge.* We used advanced LLM (e.g., Gemini-3.1-Pro) as the judge to automatically evaluate question clarity, task–tool alignment, and the validity of input–output dependencies within each reference tool chain. Only instances receiving a "Yes" judgment were retained.
- *Human Validation.* Student reviewers and domain experts then assessed the filtered samples using four predefined criteria: (1) Level Alignment, (2) Semantic Clarity, (3) Tool-Chain Coherence, and (4) Answerability. Only samples approved by a majority of the reviewers were accepted.

Based on the process described above, we construct the final OpenSciToolBench, which contains 900 questions across four difficulty levels, multiple scientific domains, and four question formats. More details about the validation protocol and dataset statistics can be found in Appendix D.

4 Methods

In this section, we introduce SciToolAgent-Evo, an ontology-aware self-evolving agent for open-world scientific tool acquisition. As illustrated in Figure 2, SciToolAgent-Evo consists of two phases: (1) *Knowledge Accumulation*. Given a set of training tasks $\mathcal{D}_{train}$, the agent performs deterministic baseline sampling and temperature-based exploratory sampling, and distills reusable knowledge from contrastive trajectories. (2) *Ontology-Aware Inference*. For a test task q_{test}, the agent retrieves memory, formulates active tool requests, and uses a LinUCB-based bandit gate to acquire tools for executable tool-chain construction.

4.1 Problem Formulation

Given a scientific tool-use task q, the agent aims to construct an executable tool chain

$$\tau = (t_1, t_2, \ldots, t_m). \tag{1}$$

It then derives the final answer via sequential execution, denoted as

$$\hat{y} = \text{Exec}(q, \tau). \tag{2}$$

Let $\mathcal{T}$ denote the complete tool space, which is partitioned into a known tool set $\mathcal{T}_K$ and an unknown tool set $\mathcal{T}_U$. Tools within $\mathcal{T}_K$ are organized as a known tool graph $G_K = (V_K, E_K)$, whereas unknown tools remain outside this graph structure:

$$\mathcal{T} = \mathcal{T}_K \cup \mathcal{T}_U, \quad \mathcal{T}_K \cap \mathcal{T}_U = \emptyset. \tag{3}$$

During each planning step, the agent dynamically decides whether to exploit known tools from G_K or explore unknown tools within $\mathcal{T}_U$. To formalize this open-world scientific tool acquisition process, we introduce the following four core components.

Definition 1 (Ontology). An ontology $o \in \mathcal{O}$ is a structured representation that characterizes the semantics, invocation constraints, and scientific applicability of a tool in the known tool graph. Formally, $o_t = (b_t, h_t)$, where b_t denotes basic attributes, including functionality, input and output types, and category; h_t denotes advanced scientific attributes, including applicable objects and scientific scenarios. To simulate realistic open-world settings, each unknown tool $u \in \mathcal{T}_U$ is initially under-specified, with only a minimal set of basic attributes exposed.

Definition 2 (Skill). A skill $k_i \in \mathcal{K}$ is a task-level tool-chain guidance unit that provides a generalizable workflow for a specific class of tasks. Each skill is defined as $k_i = (N_i, Z_i, E_i)$, where N_i denotes the skill name, Z_i is a concise description of the skill, and $E_i = (t_1, t_2, \ldots, t_m)$ represents reusable tool chains (Jiang et al. 2026).

Definition 3 (Experience). An experience $e \in \mathcal{E}$ is a non-parametric prompt that provides fine-grained guidance for tool selection. Each experience is represented as $e = (d_e, s_e, a_e, h_e)$, where d_e denotes the associated domain, s_e specifies the applicable scientific situation, a_e gives the corresponding tool-selection guidance, and $h_e \in \mathbb{R}^{d_h}$ denotes its retrieval embedding.

Definition 4 (Bandit Gate). A bandit gate is a LinUCB-based (Li et al. 2010) decision module that balances the inherent open-world exploration–exploitation trade-off. For each tool request r_j, the gate constructs an ontology-guided, four-dimensional context vector $\mathbf{x}_j = [f_j, m_j, s_j, p_j]^\top$, where f_j denotes the top-1 known-tool feasibility, m_j denotes the confidence margin between the top-2 candidates, s_j denotes the ontology alignment score, and p_j denotes the historical success prior of unknown-tool exploration.

With these definitions, we model each open-world scientific tool acquisition task as a Partially Observable Markov Decision Process (POMDP). Let q denote a task instance. The agent is equipped with a known tool graph $G_K = (V_K, E_K)$, which contains structured tool attributes and tool-level relations. At each time step t, the agent receives an observation o_t of tools retrieved from the graph, and generates an action a_t based on the current state s_t. The action corresponds to either known-tool exploitation or unknown-tool exploration. This process yields a trajectory $\xi = \big[(s_0, a_0, o_0), \ldots, (s_T, a_T, o_T)\big]$, which finally concludes with execution and summarization to produce the final answer $\hat{y}$. The objective is to construct an evolving memory $\mathcal{M} = (G_K, \mathcal{K}, \mathcal{E})$ that, when combined with the agent, executor, and summarizer, maximizes the probability of generating the correct answer: $\max_{\mathcal{M}} P(\hat{y} = y^* \mid q, \mathcal{M})$.

4.2 Knowledge Accumulation

In long-horizon tool-use tasks, a single failed trajectory is too verbose and noisy to determine whether an error arises from a policy flaw or stochastic execution noise. Therefore, without updating the backbone LLM parameters, SciToolAgent-Evo distills tool-use knowledge from contrastive rollouts.

Contrastive Rollout Generation. For each training task $q_i \in \mathcal{D}_{train}$, the agent first generates the trajectory ξ_i^{base}

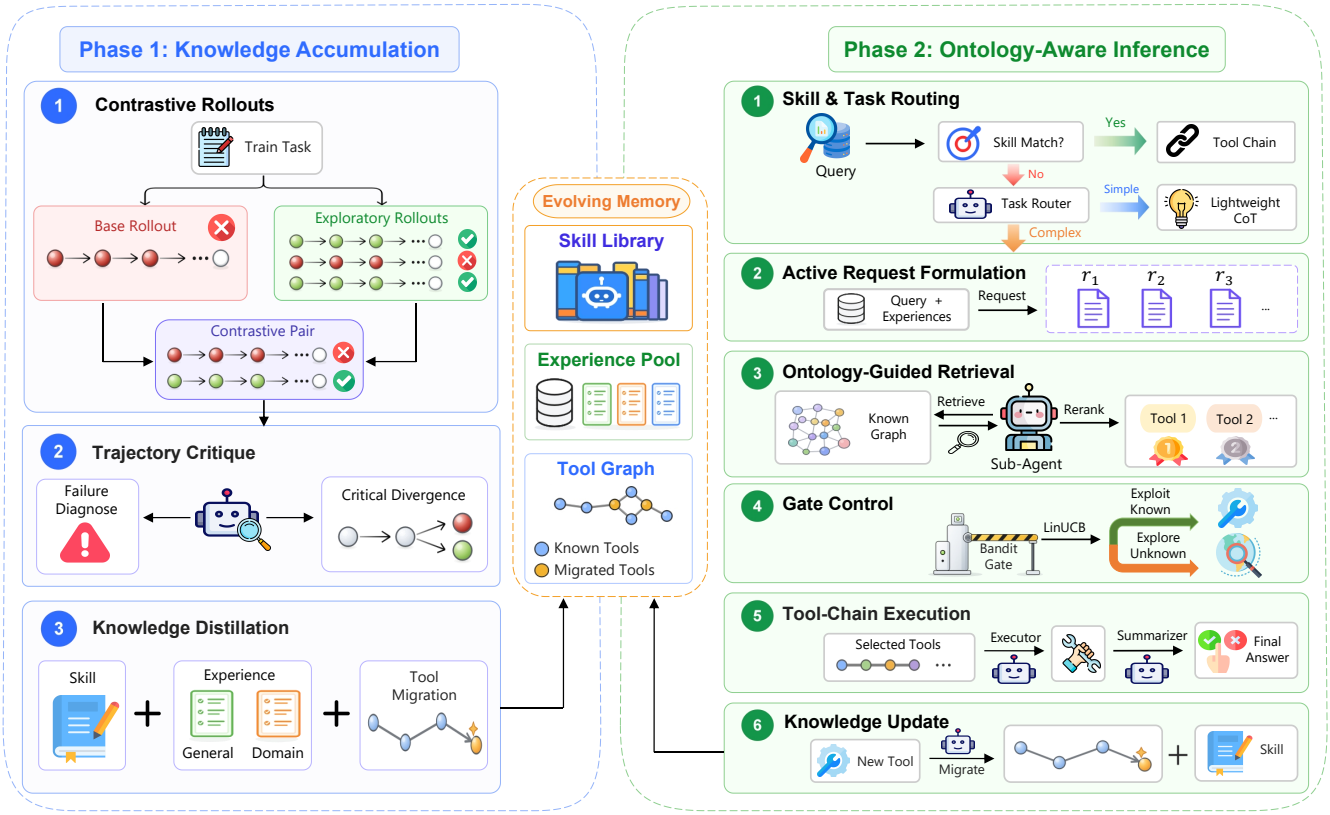


Figure 2: Overview of the SciToolAgent-Evo framework. **Phase I** (left) performs knowledge accumulation over training tasks through contrastive rollout generation, trajectory critique, and knowledge distillation. **Phase II** (right) performs ontology-aware inference through skill and task routing, active request formulation, ontology-guided retrieval, LinUCB-based gate control, tool-chain execution, and knowledge update. Together, the two phases continuously enrich a shared evolving memory, which consists of a skill library, an experience pool, and a tool graph.

at temperature T_{base}. If ξ_i^{base} succeeds, the system extracts a skill k_i abstracted from the trajectory and stores it in $\mathcal{K}$. Otherwise, the agent rolls out $N-1$ additional trajectories $\{\xi_i\}_{i=1}^{N-1}$ at different temperatures $T_{\text{exp}} = \{\gamma_n\}_{n=1}^{N-1}$. If any exploratory trajectory succeeds, the system samples one successful trajectory ξ_i^+ and constructs the contrastive pair (ξ_i^{base}, ξ_i^+).

Rollout Critique and Knowledge Distillation. Given the contrastive pair (ξ_i^{base}, ξ_i^+), the agent first identifies the critical divergence and produces a diagnostic text C_i to explain the corresponding failure reason. Based on C_i, the agent further distills two types of tool-selection experiences, namely domain-specific experiences e_i^{dom} and general experiences e_i^{gen}. Before adding them to the experience pool, the system checks whether any existing entry exhibits a cosine similarity above θ_{sim} with the candidate experience e. If so, the candidate experience is discarded; otherwise, e is added directly. Detailed prompts for skill extraction and experience distillation are provided in Appendix I.

4.3 Ontology-Aware Inference

Skill and Task Routing. Given a test task q, the agent first checks if a relevant skill can be retrieved from $\mathcal{K}$. If a matched skill is found, it executes its tool chain and summarizes the final answer. Otherwise, the agent estimates difficulty from expected tool and entity counts. Simple tasks are solved with lightweight CoT planning, while complex tasks are routed to active tool acquisition.

Active Tool Acquisition. The agent first retrieves relevant experiences to augment its context. While general experiences $\mathcal{E}_{\text{gen}}$ provide default guidance, domain-specific experiences are retrieved via semantic similarity:

$$\mathcal{E}_q = \mathcal{E}_{\text{gen}} \cup \text{TopK}(\mathcal{E}_{d_q}, q, K). \tag{4}$$

Here, d_q denotes the task domain, and K specifies the number of retrieved experiences.

Rather than passively selecting tools based on the raw query, the agent actively formulates tool requests via ontology-guided reasoning. Given q and $\mathcal{E}_q$, it generates a set of active tool requests $R_q = \{r_1, r_2, \ldots, r_l\}$, where each r_j is a concise requirement description for one required tool. We justify this design in Appendix F.

For each request r_j, SciToolAgent-Evo identifies candidate tools within the known tool graph G_K. Let $t \in V_K$ denote

Method	Level-1		Level-2		Level-3		Level-4		Overall	
	Tool	Ans.	Tool	Ans.	Tool	Ans.	Tool	Ans.	Tool	Ans.
GPT-5.4-mini										
Direct	52.66±1.69	51.43±1.34	46.31±1.49	43.68±0.74	24.71±1.66	21.18±3.12	23.03±2.79	21.05±2.27	38.11±0.86	35.87±0.42
CoT	75.24±0.00	73.33±0.95	66.32±2.98	64.56±1.61	38.83±1.66	36.08±2.04	28.29±4.65	25.43±3.72	54.44±0.59	52.17±0.89
RAG	64.76±0.95	62.54±1.46	57.89±3.16	55.44±3.22	31.76±2.04	28.23±3.11	19.74±2.63	18.42±2.63	45.70±0.48	43.31±1.37
ReAct	74.06±0.67	71.31±0.67	58.95±2.98	54.21±3.72	30.58±1.17	25.88±0.83	23.68±3.72	19.74±1.32	49.45±1.56	45.26±0.64
Reflexion	78.30±1.34	76.19±1.34	64.39±4.87	60.34±3.38	40.10±3.02	35.29±2.36	26.32±3.72	20.62±2.01	54.70±2.16	50.69±1.39
SciToolAgent	53.33±2.69	51.91±1.98	48.42±1.16	47.37±3.01	33.94±2.77	30.58±2.77	17.11±1.86	14.91±3.04	39.85±0.70	37.90±1.73
SciToolAgent-Evo	**85.71**±0.00	**83.49**±0.55	**76.84**±3.01	**75.26**±2.23	**58.82**±1.65	**56.32**±1.17	**36.84**±4.13	**32.89**±3.48	**66.76**±0.74	**64.28**±0.85
Gemini-3-flash										
Direct	43.34±0.67	40.64±0.67	28.42±1.48	27.37±1.05	17.06±0.83	16.08±0.68	6.58±0.00	6.14±0.76	25.48±0.39	24.10±1.05
CoT	57.14±1.34	56.51±1.61	43.16±3.08	42.45±3.08	30.59±1.66	29.80±0.68	26.32±1.86	24.12±1.86	40.72±0.55	39.70±0.64
RAG	65.71±0.96	62.86±0.00	47.37±1.05	45.26±2.11	26.27±2.96	23.92±1.80	17.11±0.00	15.35±0.76	41.37±0.64	39.06±0.83
ReAct	75.24±1.91	72.07±1.46	58.78±2.73	53.68±2.23	41.51±3.69	36.47±3.12	27.63±0.76	23.68±0.00	52.94±2.27	48.66±1.12
Reflexion	**84.43**±0.95	80.31±2.40	67.17±2.87	63.51±4.26	54.44±3.60	48.24±1.36	34.21±3.95	29.82±1.52	62.25±1.37	57.71±1.21
SciToolAgent	63.81±2.43	62.54±1.46	53.58±3.28	53.16±2.23	43.53±0.47	41.76±2.49	22.76±4.93	21.05±3.95	47.65±2.45	46.44±2.10
SciToolAgent-Evo	83.81±0.95	**82.22**±1.32	**82.11**±2.10	**79.65**±2.10	**68.24**±5.43	**65.09**±3.78	**35.53**±2.79	**33.55**±0.93	**69.53**±2.77	**67.26**±0.96

Table 1: Evaluation results on **OpenSciToolBench** with **GPT-5.4-mini** and **Gemini-3-flash** backbones. Results are averaged over 3 independent runs and reported as mean ± standard deviation. The best result is **bolded**, and the second best is underlined. Tool and Ans. indicate tool-use and final answer accuracy.

a tool node and $S(r_j, t)$ represent its semantic similarity to the request. The initial retrieval set is formulated as:

$$C_j^0 = \text{TopK}\{t \mid S(r_j, t), \forall t \in V_K\}. \quad (5)$$

After that, graph exploration and combination reranking refine C_j^0 into the final candidate set C_j.

The sub-agent then utilizes C_j to produce the 4D context vector $\mathbf{x}_j = [f_j, m_j, s_j, p_j]^\top \in \mathbb{R}^4$ for the bandit gate. The gate selects between known-tool exploitation and unknown-tool exploration using a LinUCB policy:

$$a_j = \arg\max_{a \in \{\texttt{a1}, \texttt{a2}\}} \left(\hat{\boldsymbol{\theta}}_a^\top \mathbf{x}_j + \alpha \sqrt{\mathbf{x}_j^\top A_a^{-1} \mathbf{x}_j} \right), \quad (6)$$

where `a1` and `a2` denote known-tool exploitation and unknown-tool exploration, respectively. Here, $\hat{\boldsymbol{\theta}}_a = A_a^{-1}\mathbf{b}_a$ estimates the reward of action a, and α controls the exploration strength. For exploitation, the agent selects the top-ranked candidate from C_j; for exploration, it infers tool utility from sparse attributes and retrieves the most compatible candidate from $\mathcal{T}_U$. The action is applied iteratively until a complete tool chain $\hat{\tau}$ is constructed. Finally, the Executor runs $\hat{\tau}$ sequentially, and the Summarizer produces $\hat{y}$.

Knowledge Update. Inspired by human learning, successful exploration is consolidated into reusable knowledge. When a rollout successfully utilizes an unknown tool, the agent completes its ontology o_t from the interaction history and migrates it into the known tool graph G_K. The same execution signal provides a binary reward $r \in \{0, 1\}$ for updating the parameters of the selected action a: $A_a \leftarrow A_a + \mathbf{x}_j\mathbf{x}_j^\top$, $\mathbf{b}_a \leftarrow \mathbf{b}_a + r\mathbf{x}_j$. In addition, the resulting trajectory is abstracted into a skill k_{new} and stored in $\mathcal{K}$. Together, these updates establish a closed self-evolving loop, enabling the agent to adapt continuously in open-world environments.

5 Experiments

5.1 Evaluation Benchmarks and Metrics

We conduct a comprehensive evaluation of SciToolAgent-Evo on two scientific tool-use benchmarks:

- **OpenSciToolBench**: Contains 900 scientific tool-use questions to assess capabilities ranging from explicit tool invocation to open-ended research strategy formulation.
- **SciToolEval** (Ding et al. 2025): Features 531 questions comprising 152 single-tool and 379 multi-tool tasks, covering molecular property prediction, protein analysis, and materials retrieval.

For both benchmarks, we apply **Tool Accuracy** (measuring tool-chain alignment with the reference) and **Answer Accuracy** (assessing final-answer correctness) as the final metrics.

5.2 Baselines

We evaluate SciToolAgent-Evo under two proprietary LLM backbones, GPT-5.4-mini (OpenAI 2025) and Gemini-3-flash (Team et al. 2023), and compare it against various representative methods: **Direct** generates the final answer directly without producing intermediate reasoning; **CoT** (Wei et al. 2022) produces an intermediate reasoning trajectory before answering; **RAG** (Lewis et al. 2020) retrieves relevant external knowledge to augment answer generation; **ReAct** (Yao et al. 2023) and **Reflexion** (Shinn et al. 2023) serve as interactive tool-use methods, relying on the Thought-Action-Observation loop and trajectory-level self-feedback for plan-

Method	Level-1		Level-2		Overall	
	Tool	Ans.	Tool	Ans.	Tool	Ans.
GPT-5.4-mini						
Direct	53.29±0.93	51.75±1.24	46.43±1.68	45.24±1.98	49.01±0.70	47.69±0.81
CoT	86.84±1.86	85.53±1.32	72.62±2.81	71.43±2.38	77.97±2.45	76.73±1.73
RAG	76.32±1.32	74.12±0.76	48.94±1.99	48.41±2.10	59.24±1.51	58.09±1.51
ReAct	83.56±3.48	82.45±2.74	69.05±3.46	68.25±3.46	74.51±2.70	73.59±2.23
Reflexion	87.28±2.28	86.39±2.01	70.63±1.68	68.78±1.99	76.89±1.24	75.41±2.20
SciToolAgent	62.50±2.79	61.18±2.79	46.43±1.68	45.24±1.12	52.48±2.10	51.24±1.75
SciToolAgent-Evo	**90.13**±0.93	**88.60**±0.76	**80.16**±3.37	**79.37**±3.37	**83.91**±1.75	**82.84**±1.87
Gemini-3-flash						
Direct	55.26±1.86	53.95±1.86	25.00±2.80	25.00±2.80	36.37±2.45	35.89±2.45
CoT	87.50±0.93	86.84±0.76	51.98±2.81	50.79±2.25	65.34±2.10	64.35±1.40
RAG	78.07±3.31	76.75±3.31	53.44±1.21	52.38±1.59	62.71±1.59	61.55±1.87
ReAct	85.53±2.78	85.09±1.52	69.84±1.12	69.04±0.57	75.74±1.75	75.08±0.76
Reflexion	**88.82**±1.32	**88.16**±1.42	65.48±3.96	64.68±2.71	74.26±1.51	73.51±1.03
SciToolAgent	71.06±3.72	70.39±2.79	55.17±2.76	54.81±2.18	61.14±1.40	60.67±0.33
SciToolAgent-Evo	87.72±0.88	86.84±1.87	**73.41**±1.68	**72.62**±1.32	**78.79**±1.40	**77.97**±1.75

Table 2: Evaluation results on **SciToolEval** with **GPT-5.4-mini** and **Gemini-3-flash** backbones. Results are averaged over 3 independent runs and reported as mean ± standard deviation. The best result is **bolded**, and the second best is underlined. Tool and Ans. indicate tool-use and final answer accuracy.

ning respectively; and **SciToolAgent** (Ding et al. 2025) leverages a tool knowledge graph for tool retrieval, tool-chain planning, and execution. More details of these baselines are presented in Appendix H.

5.3 Implementation Details

We employ locally deployed Qwen3-Embedding-0.6B and Qwen3-1.7B (Yang et al. 2025) as our retrieval model and reranking model, respectively. For all baselines, we use the same open-world split, with 50% of the toolset designated as known and the remaining 50% as unknown. During accumulation, we sample $K = 4$ trajectories for each task, including one deterministic baseline trajectory with $T_{\text{base}} = 0$ and three exploratory trajectories with $T_{\text{exp}} \in \{0.2, 0.5, 0.8\}$. At test time, we use a LinUCB-based bandit gate with $\alpha_0 = 0.8$, decay rate 0.01, $\alpha_{\min} = 0.2$, and a 20-step failure window to scale α within $[0.7, 1.3]$. More implementation details are provided in Appendix G.

5.4 Main Results

Tables 1 and 2 show the results across two benchmarks and two backbone models. Our method demonstrates substantial improvements over all baselines. Compared with the runner-up results, SciToolAgent-Evo improves overall answer accuracy on OpenSciToolBench by 12.11 and 9.55 points with GPT-5.4-mini and Gemini-3-flash, respectively. On SciToolEval, the corresponding gains are 6.11 and 2.89 points. Notably, SciToolAgent-Evo exceeds SciToolAgent by a striking 17% to 31% margin across all benchmarks. This gain reflects a key distinction: SciToolAgent remains constrained by a fixed known-tool graph, whereas SciToolAgent-Evo dynamically balances exploration and exploitation and consolidates acquired knowledge to enable reliable acquisition of unknown tools, yielding stronger open-world adaptability.

From a fine-grained analysis across difficulty levels, we observe that while the performance gap is marginal on simpler tasks (Level-1), SciToolAgent-Evo demonstrates a significant advantage in solving complex multi-tool tasks. For instance, on OpenSciToolBench with GPT-5.4-mini, SciToolAgent-Evo achieves 56.32% answer accuracy on Level-3, surpassing Reflexion and ReAct by more than 20 and 30 points. Empirically, we observe that in-context learning methods, such as CoT, ReAct, and Reflexion, rely heavily on local step-wise reasoning and lack a stable global strategy for long tool-chain planning. In open tool spaces with vast and initially unobserved candidates, this deficiency inevitably incurs high exploration costs and degrades tool-planning accuracy. SciToolAgent-Evo instead maintains an evolving memory, reducing exploration costs and guaranteeing robust adaptation in open-world environments.

5.5 Ablation Study

Component Ablation. We conduct a systematic ablation study on OpenSciToolBench with GPT-5.4-mini to analyze the contribution of each component. As shown in Table 3, we obtain the following results: (1) Removing knowledge accumulation causes a 14.87-point drop, showing its necessity for robust adaptation in open-world scientific tool settings. (2) Disabling unknown tool migration entirely leads to the largest degradation. Conversely, disabling it only during inference causes a much smaller drop, underscoring the necessity of continual tool migration. (3) Excluding skills and experi-

Method	Level-1	Level-2	Level-3	Level-4	Overall
w/o Knowledge Accumulation	80.00 ↓5.71	57.89 ↓18.95	45.88 ↓14.93	14.47 ↓22.37	52.35 ↓14.87
w/o Tool Migration	78.10 ↓7.61	62.11 ↓14.73	41.17 ↓19.64	13.16 ↓23.68	51.53 ↓15.69
w/o Inference Migration	81.90 ↓3.81	72.63 ↓4.21	56.47 ↓4.34	35.53 ↓1.31	63.71 ↓3.51
w/o Skill	83.81 ↓1.90	73.68 ↓3.16	54.12 ↓6.69	35.53 ↓1.31	63.99 ↓3.23
w/o Experience	85.71 ↓0.00	75.79 ↓1.05	57.65 ↓3.16	34.21 ↓2.63	65.65 ↓1.57
Ours - Full Pipeline	85.71	76.84	60.81	36.84	67.22

Table 3: Ablation study on **OpenSciToolBench**. All variants use GPT-5.4-mini as the backbone. We remove key components to assess their contributions, and ↓ denotes the absolute performance drop from the full pipeline.

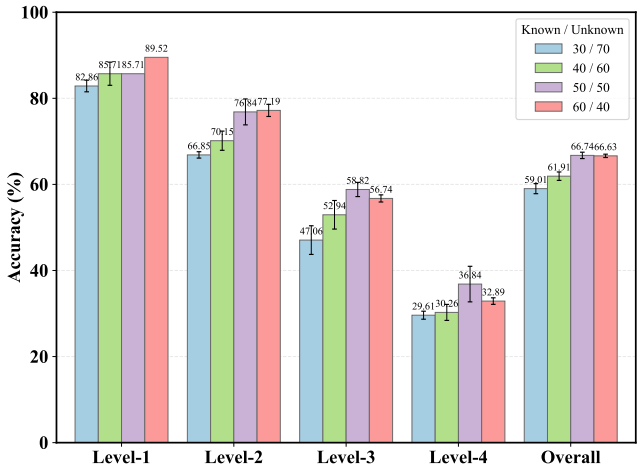


Figure 3: Performance sensitivity to known/unknown tool splits on **OpenSciToolBench** with GPT-5.4-mini.

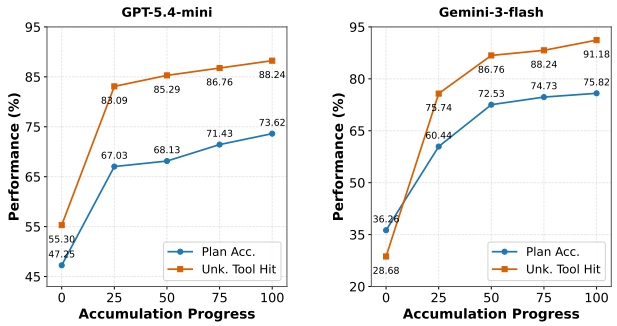


Figure 4: Performance comparison across knowledge accumulation stages on **OpenSciToolBench** with GPT-5.4-mini and Gemini-3-flash, measured by Plan Acc. (planning accuracy) and Unk. Tool Hit (hit rate on initially unknown tools).

ences degrades performance by 3.23 and 1.57 points, respectively, demonstrating that both knowledge forms are critical. These results indicate that each component addresses a distinct challenge in open-world scientific tool use, and their integration enables continual refinement of tool memory and acquisition policy. Further fine-grained ablation results for the inference pipeline are reported in Appendix B1.

Known/Unknown Split Sensitivity. To further validate the robustness of our method, we evaluate SciToolAgent-Evo on OpenSciToolBench across four known/unknown tool splits using GPT-5.4-mini. As shown in Figure 3, overall accuracy generally improves as the known-tool ratio grows, rising from 59.01% under the 30/70 split to 66.63% under the 60/40 split. This suggests that a richer known-tool graph reduces open-world exploration difficulty and stabilizes toolchain planning. Crucially, even when 70% of tools are initially unknown, SciToolAgent-Evo still achieves a competitive 59.01% overall accuracy. This result demonstrates the framework's robust tool-acquisition capability in challenging open-world settings.

5.6 Knowledge Accumulation Dynamics

To investigate how knowledge accumulation affects performance, we evaluate SciToolAgent-Evo at different accumulation stages on OpenSciToolBench with GPT-5.4-mini and Gemini-3-flash. As shown in Figure 4, both Planning Accuracy and Unknown Tool Hit Rate improve consistently as the process advances. These gains stem from the key mechanisms of the accumulation process, including contrastive experience extraction, reusable skill distillation, and dynamic tool migration. Notably, both metrics grow rapidly from 0% to 25% progress and then improve more smoothly, suggesting that a limited amount of data is sufficient for the agent to establish robust priors for open-world adaptation. Detailed statistics are provided in Appendix B2.

6 Conclusion

In this work, we propose an ontology-aware self-evolving agent for open-world scientific tool acquisition. SciToolAgent-Evo follows a two-stage design: during accumulation, it distills reusable knowledge from contrastive trajectories; during inference, it uses active requests and a LinUCB-based bandit gate to balance tool exploration and exploitation. By leveraging an adaptive memory of skills, experiences, and an ontologized tool graph, SciToolAgent-Evo enables agents to continually expand their capabilities in open-world scientific environments. Moreover, we introduce OpenSciToolBench, a scientific tool-use benchmark with 900 tasks across four difficulty levels. Extensive experiments demonstrate that SciToolAgent-Evo outperforms strong baselines, indicating its robustness and generalization.

References

Chaudhari, A.; Ock, J.; Barati Farimani, A.; and et al. 2026. Modular large language model agents for multi-task computational materials science. *Communications Materials*, 7(1): 131.

Chen, Z.; Chen, S.; Ning, Y.; Zhang, Q.; Wang, B.; Yu, B.; Li, Y.; Liao, Z.; Wei, C.; Lu, Z.; et al. 2025. Scienceagentbench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *International Conference on Learning Representations*, volume 2025, 96934–96990.

Ding, K.; Yu, J.; Huang, J.; Yang, Y.; Zhang, Q.; and Chen, H. 2025. SciToolAgent: a knowledge-graph-driven scientific agent for multitool integration. *Nature Computational Science*, 5(10): 962–972.

Ghareeb, A. E.; Chang, B.; Mitchener, L.; Yiu, A.; Szostkiewicz, C. J.; Shved, D.; Gyimesi, G. J.; Laurent, J. M.; Wright, S. M.; Razzak, M. T.; et al. 2026. A multi-agent system for automating scientific discovery. *Nature*, 1–3.

Hess, K.; Jones, B.; Carlock, D.; and Walkup, J. R. 2009. Cognitive rigor: Blending the strengths of Bloom's taxonomy and Webb's depth of knowledge to enhance classroom-level processes.

Jiang, G.; Su, Z.; Qu, X.; and Fung, Y. R. 2026. Xskill: Continual learning from experience and skills in multimodal agents. *arXiv:2603.12056*.

Jin, Q.; Yang, Y.; Chen, Q.; and Lu, Z. 2024. GeneGPT: augmenting large language models with domain tools for improved access to biomedical information. *Bioinformatics*, 40(2): btae075.

Kang, Y.; and Kim, J. 2024. ChatMOF: an artificial intelligence system for predicting and generating metal-organic frameworks using large language models. *Nature communications*, 15(1): 4705.

Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33: 9459–9474.

Li, L.; Chu, W.; Langford, J.; and Schapire, R. E. 2010. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th international conference on World wide web*, 661–670.

Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. Api-Bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, 3102–3116.

Liu, X.; Yu, H.; Zhang, H.; Xu, Y.; Lei, X.; Lai, H.; Gu, Y.; Ding, H.; Men, K.; Yang, K.; et al. 2024. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, volume 2024, 52989–53046.

Lu, J.; Holleis, T.; Zhang, Y.; Aumayer, B.; Nan, F.; Bai, H.; Ma, S.; Ma, S.; Li, M.; Yin, G.; et al. 2025. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 1160–1183.

M. Bran, A.; Cox, S.; Schilter, O.; Baldassari, C.; White, A. D.; and Schwaller, P. 2024. Augmenting large language models with chemistry tools. *Nature machine intelligence*, 6(5): 525–535.

Ma, Y.; Gou, Z.; Hao, J.; Xu, R.; Wang, S.; Pan, L.; Yang, Y.; Cao, Y.; and Sun, A. 2024a. Sciagent: Tool-augmented language models for scientific reasoning. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, 15701–15736.

Ma, Z.; Huang, W.; Zhang, J.; Gupta, T.; and Krishna, R. 2024b. m & m's: A benchmark to evaluate tool-use for m ulti-step m ulti-modal tasks. In *European Conference on Computer Vision*, 18–34. Springer.

Mekala, D.; Weston, J. E.; Lanchantin, J.; Raileanu, R.; Lomeli, M.; Shang, J.; and Dwivedi-Yu, J. 2024. Toolverifier: Generalization to new tools via self-verification. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 5026–5041.

Min, B.; Ross, H.; Sulem, E.; Veyseh, A. P. B.; Nguyen, T. H.; Sainz, O.; Agirre, E.; Heintz, I.; and Roth, D. 2023. Recent advances in natural language processing via large pre-trained language models: A survey. *ACM Computing Surveys*, 56(2): 1–40.

OpenAI. 2025. GPT-5 System Card. Technical report, OpenAI.

Paranjape, B.; Lundberg, S.; Singh, S.; Hajishirzi, H.; Zettlemoyer, L.; and Ribeiro, M. T. 2023. Art: Automatic multi-step reasoning and tool-use for large language models. *arXiv:2303.09014*.

Patil, S. G.; Zhang, T.; Wang, X.; and Gonzalez, J. E. 2024. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37: 126544–126565.

Qian, C.; Acikgoz, E. C.; He, Q.; Wang, H.; Chen, X.; Hakkani-Tur, D.; Tur, G.; and Ji, H. 2026. Toolrl: Reward is all tool learning needs. *Advances in Neural Information Processing Systems*, 38: 105523–105553.

Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; et al. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, volume 2024, 9695–9717.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36: 68539–68551.

Shen, Y.; Song, K.; Tan, X.; Li, D.; Lu, W.; and Zhuang, Y. 2023. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36: 38154–38180.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36: 8634–8652.

Song, Y.; Xiong, W.; Zhu, D.; Wu, W.; Qian, H.; Song, M.; Huang, H.; Li, C.; Wang, K.; Yao, R.; et al. 2023. RestGPT: Connecting large language models with real-world restful apis. *arXiv:2306.06624*.

Tang, Q.; Deng, Z.; Lin, H.; Han, X.; Liang, Q.; Cao, B.; and Sun, L. 2023. Toolalpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv:2306.05301*.

Tang, Y.; Feng, K.; Wang, Y.; Chen, Z.; Lv, C.; Yu, G.; Zhang, Q.; Ding, K.; and Chen, H. 2025. Learning an Efficient Multi-Turn Dialogue Evaluator from Multiple LLM Judges. *arXiv:2508.00454*.

Team, G.; Anil, R.; Borgeaud, S.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; Millican, K.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv:2312.11805*.

Trivedi, H.; Khot, T.; Hartmann, M.; Manku, R.; Dong, V.; Li, E.; Gupta, S.; Sabharwal, A.; and Balasubramanian, N. 2024. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 16022–16076.

Vemprala, S. H.; Bonatti, R.; Bucker, A.; and Kapoor, A. 2024. ChatGPT for robotics: Design principles and model abilities. *Ieee Access*, 12: 55682–55696.

Wang, H.; Qian, C.; Zhong, W.; Chen, X.; Qiu, J.; Huang, S.; Jin, B.; Wang, M.; Wong, K.-F.; and Ji, H. 2025a. OTC: Optimal tool calls via reinforcement learning. *arXiv–2504*.

Wang, L.; Ma, C.; Feng, X.; Zhang, Z.; Yang, H.; Zhang, J.; Chen, Z.; Tang, J.; Chen, X.; Lin, Y.; et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6): 186345.

Wang, R.; Han, X.; Ji, L.; Wang, S.; Baldwin, T.; and Li, H. 2025b. Toolgen: Unified tool retrieval and calling via generation. In *International Conference on Learning Representations*, volume 2025, 73473–73498.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q. V.; Zhou, D.; et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35: 24824–24837.

Wu, H.; He, Z.; Zhang, X.; Yao, X.; Zheng, S.; Zheng, H.; and Yu, B. 2024. Chateda: A large language model powered autonomous agent for eda. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43(10): 3184–3197.

Yang, A.; Li, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Gao, C.; Huang, C.; Lv, C.; et al. 2025. Qwen3 technical report. *arXiv:2505.09388*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K. R.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.

Yu, J.; Tang, Y.; Feng, K.; Liang, L.; Zhang, Q.; Ding, K.; and Chen, H. 2026. SciCuEval: A comprehensive dataset for evaluating scientific context understanding in large language models. *Scientific Data*.

A Limitations

Despite the efficacy of SciToolAgent-Evo in open-world scientific tool acquisition, several limitations remain. First, our evaluation is conducted in a purely text-based setting, which abstracts away critical multi-modal aspects of real scientific workflows and leaves multi-modal tool adaptation underexplored. For future work, we will evaluate our framework on multi-modal scientific tasks and explore dynamic memory updating and active unlearning strategies for more durable open-world tool adaptation.

B Additional Results

In this section, we demonstrate the effectiveness of SciToolAgent-Evo through additional experiments. We first analyze fine-grained ablations (Appendix B1) and knowledge accumulation dynamics (Appendix B2). We then evaluate out-of-domain generalization (Appendix B3). Finally, we investigate inference efficiency (Appendix B4).

B1 Fine-Grained Ablation Study

In Section 5.5, we reported the overall component-level ablation results (Table 3). In this section, we conduct a fine-grained ablation of the ontology-aware inference pipeline using GPT-5.4-mini as the backbone to assess the contribution of each module. Table A1 yields the following observations: (1) Disabling the Task Router reduces Level-1 and Level-2 performance by 8.57 and 10.52 points, respectively, suggesting that it avoids unnecessary reasoning on simpler tasks. (2) Removing Active Request or replacing it with Retrieval Rewriting substantially degrades performance, indicating that requirement-conditioned retrieval better captures task-specific tool needs than raw-query matching. (3) Excluding Reranking degrades overall performance by 4.06 points, confirming the value of candidate refinement. (4) LinUCB outperforms both the Fixed Threshold Gate and the LLM-based Gate, indicating a more effective balance between known-tool exploitation and unknown-tool exploration. Overall, these results confirm that each component is necessary for an effective ontology-aware inference pipeline.

B2 Detailed Knowledge Accumulation Statistics

Table A2 presents detailed statistics of the knowledge accumulation process on OpenSciToolBench with GPT-5.4-mini. We report results at five checkpoints, tracking the number of migrated unknown tools, the number of accumulated skills, the skill-based success rate, planning accuracy, and the hit rate on initially unknown tools. As accumulation progresses, the numbers of acquired tools and accumulated skills increase steadily, while planning accuracy and the unknown-tool hit rate improve by 26.37 and 32.94 points, respectively. These results further demonstrate the importance of maintaining an evolving memory throughout knowledge accumulation.

B3 Out-of-Domain Generalization

While our primary focus remains on enhancing agent performance in open-world scientific tool acquisition, we also acknowledge the potential of SciToolAgent-Evo in out-of-domain settings. To validate the generalizability of our approach, we conducted additional experiments on m&m's (Ma et al. 2024b), a multi-step multimodal tool-use benchmark, and RestBench (Song et al. 2023), a benchmark for planning and executing RESTful API calls. Following the official evaluation protocols and metrics of each benchmark, we used GPT-5.4-nano as the backbone model for all methods and compared against the same baselines introduced in Section 5.2 to ensure a fair comparison.

As shown in Table A3, SciToolAgent-Evo achieves the highest tool-F1 (80.82) and argname-F1 (79.04) on m&m's, outperforming prompting-based (Direct and CoT), retrieval-based (RAG and SciToolAgent), and interactive (ReAct and Reflexion) baselines. On RestBench, SciToolAgent-Evo achieves the best overall performance on both TMDB and Spotify, particularly in success rate and completion percentage. Overall, these results demonstrate that SciToolAgent-Evo generalizes effectively to broader tool-use domains.

B4 Inference Efficiency

We further compare the inference efficiency of SciToolAgent-Evo with representative baselines on OpenSciToolBench using GPT-5.4-mini. As shown in Table A4, although SciToolAgent-Evo introduces additional overhead over SciToolAgent due to modules such as evolving-memory retrieval and active request formulation, it uses substantially fewer total tokens and has lower API cost than ReAct and Reflexion. It also achieves the lowest output-token usage and relatively low inference latency. Overall, SciToolAgent-Evo maintains strong inference efficiency while enhancing open-world tool acquisition.

C Task Levels and Prompts

In this section, we provide detailed definitions of the four levels in OpenSciToolBench and present the prompt templates below. The distinctions among these levels are inspired by the Cognitive Rigor Matrix (Hess et al. 2009), with task difficulty characterized jointly by knowledge mapping, reasoning depth, constraint integration, and decision complexity.

Level-1 (1–2 tools) consists of explicit invocation tasks, where the question explicitly provides the tool name or clear hints to the required tool. These tasks typically involve a single tool call or a short tool chain and focus on basic data retrieval for a single scientific entity.

Prompt for Generating Level-1 Questions

System Prompt:
You are a science expert.

User Prompt:
Please generate one scientific tool-use question based on the given candidate tools. The question type should be one of Question & Answer, Multiple-Choice, Content Completion, and True/False.

Candidate Tools:

Method	Level-1	Level-2	Level-3	Level-4	Overall
w/o Task Router	77.14 ↓8.57	66.32 ↓10.52	61.18 ↑0.37	36.84 0.00	62.05 ↓5.17
w/o Active Request	81.90 ↓3.81	65.26 ↓11.58	43.53 ↓17.28	23.52 ↓13.32	56.20 ↓11.02
Retrieval Rewriting	82.86 ↓2.85	66.32 ↓10.52	49.41 ↓11.40	26.32 ↓10.52	58.73 ↓8.49
Query Decomposition	82.86 ↓2.85	76.84 0.00	57.65 ↓3.16	35.53 ↓1.31	65.38 ↓1.84
w/o Reranking	84.76 ↓0.95	69.47 ↓7.37	54.12 ↓6.69	35.53 ↓1.31	63.16 ↓4.06
Fixed Threshold Gate	85.71 0.00	74.74 ↓2.10	58.82 ↓1.99	30.26 ↓6.58	64.82 ↓2.40
LLM-based Gate	86.67 ↑0.96	75.79 ↓1.05	57.65 ↓3.16	32.89 ↓3.95	65.65 ↓1.57
Ours - Full Pipeline	85.71	76.84	60.81	36.84	67.22

Table A1: Fine-grained ablation study of ontology-aware inference modules on **OpenSciToolBench**. All variants use GPT-5.4-mini as the backbone. We remove or replace key components to assess their contributions. ↓ and ↑ denote the absolute performance drop and improvement relative to the full pipeline, respectively, while gray values indicate no change.

Progress	#Acquired Tools	#Skills	Skill Hit	Plan Acc.	Unk. Tool Hit
0%	0	0	1.20	47.25	55.30
25%	39	80	4.40	67.03	83.09
50%	62	147	7.69	68.13	85.29
75%	69	214	6.59	71.43	86.76
100%	76	268	12.09	73.62	88.24

Table A2: Performance evolution across knowledge accumulation stages on **OpenSciToolBench** with GPT-5.4-mini. Metrics include the number of migrated tools (#Acquired Tools), accumulated skills (#Skills), Skill-based success rate (Skill Hit), planning accuracy (Plan Acc.), and hit rate on initially unknown tools (Unk. Tool Hit).

Method	m&m's		RestBench				
			TMDB			Spotify	
	tool-F1 ↑	argname-F1 ↑	Success% ↑	CP% ↑	Δ Sol. Len. ↓	CP% ↑	Δ Sol. Len. ↓
Direct	61.77	63.23	30.0	34.0	0.42	34.48	0.76
CoT	71.58	71.46	<u>44.0</u>	<u>48.0</u>	−0.06	44.83	−0.07
RAG	<u>71.62</u>	<u>72.26</u>	41.0	44.0	−0.10	46.56	**−0.36**
ReAct	69.65	70.31	42.0	46.0	0.56	48.28	0.48
Reflexion	70.17	70.81	40.0	46.0	1.28	<u>51.72</u>	0.83
SciToolAgent	65.48	67.92	40.0	42.0	**−0.24**	48.28	<u>−0.14</u>
SciToolAgent-Evo	**80.82**	**79.04**	**54.0**	**62.0**	<u>−0.13</u>	**53.45**	**−0.36**

Table A3: Comparison of model performance on **m&m's** and **RestBench** using GPT-5.4-nano as the backbone. The best and second-best results in each column are bolded and underlined, respectively.

{tools_str}

Task Objective:
Generate a basic instruction-level task. The question should focus on a single substance and use standard scientific terminology to clearly indicate the capability corresponding to a single tool or a very short tool chain.

[RULE]:
1. The question must evaluate explicit instruction following and clearly require a specific scientific operation.
2. tool_path should contain a single tool or a very short tool chain arranged in the correct execution order.
3. The question may not describe the solving steps or

Method	LLM Calls ↓	Input Tokens ↓	Output Tokens ↓	Total Tokens ↓	Latency (s) ↓	API Cost / Task ($) ↓
ReAct	8.2	72,115.85	427.85	72,543.7	27.9	0.056
Reflexion	16.1	149,435.95	931.15	150,367.1	44.7	0.116
SciToolAgent	**4.7**	**6,859.45**	441.05	**7,300.5**	**17.2**	**0.007**
SciToolAgent-Evo	11.3	21,711.35	**387.8**	22,099.15	26.4	0.018

Table A4: Inference cost comparison on **OpenSciToolBench** with GPT-5.4-mini. All reported results are averaged across tasks, with the best result bolded and the second-best underlined.

reveal any tool outputs, numerical results, or answer hints.
4. The Parameter field should contain only the substance name or ID selected in the question.
5. Ensure natural, diverse phrasing and avoid template-like wording.
Strictly output the response in **valid JSON** format only. The required **JSON** format is as follows:

```
{
  "tool_path": ["tool_selected", "xxx"],
  "question": "question text",
  "Parameter": "substance name"
}
```

Level-2 (2–4 tools) consists of property analysis tasks, where the question focuses on a single or a small number of target properties. These tasks typically involve short tool chains for property retrieval, direct computation, or comparison, requiring the model to select appropriate tools and make direct judgments based on their outputs.

Prompt for Generating Level-2 Questions

System Prompt:
You are a science expert.

User Prompt:
Please generate one scientific tool-use question based on the given candidate tools. The question type should be one of Question & Answer, Multiple-Choice, Content Completion, and True/False.

Candidate Tools:
{tools_str}

Task Objective:
Generate a terminology-level task. The question should use standard scientific terminology to imply the target property to be evaluated. It typically involves a short tool chain from entity representation conversion to property computation, comparison, or filtering over a few substances, requiring the model to infer the necessary tool capabilities from the property requirements.

[RULE]:
1. The question must evaluate implicit grounding from scientific terminology to a tool-capability chain.
2. tool_path should contain a short tool chain arranged in the correct execution order, supporting entity representation conversion, property computation, or property comparison.
3. The question must not directly mention any tool name, but should use standard scientific terminology to clearly imply the target property to be evaluated; it must not describe the solving steps or reveal any tool outputs, numerical results, or answer hints.
4. The Parameter field should contain only the referenced substance names or IDs, with a consistent format.
5. The question must not use comparison criteria that are unnatural in real scientific queries, such as median; instead, it should use more realistic scientific constraints such as maximum/minimum values, threshold-based judgments, or structural inclusion.
6. Ensure natural, diverse phrasing and avoid template-like wording.

Strictly output the response in **valid JSON** format only. The required **JSON** format is as follows:

```
{
  "tool_path": ["tool_selected", "xxx"],
  "question": "question text",
   "Parameter": ["substance name 1", "substance name 2", "xxx"]
}
```

Level-3 (3–6 tools) consists of constraint reasoning tasks, where the question specifies multiple candidate entities, multi-dimensional interrelated properties, and explicit scientific constraints. These tasks require integrating information from multiple sources and performing constraint-aware reasoning to infer latent relationships and identify the most appropriate candidate.

Prompt for Generating Level-3 Questions

System Prompt:
You are a science expert.

User Prompt:
Please generate one scientific tool-use question based on the given candidate tools. The question type should be one of Question & Answer, Multiple-Choice, Content Completion, and True/False.

Candidate Tools:
{tools_str}

Task Objective:
Generate an expert decision-level task. The question should provide multiple candidate objects and multiple scientific constraints. It typically requires cross-tool and cross-attribute integration over multiple substances, requiring the model to perform candidate selection and execute a relatively long tool chain under multiple constraints.

[RULE]:
1. The question must evaluate deep scientific reasoning, including property derivation, comprehensive threshold-based judgment, candidate filtering, or selection of the optimal object satisfying all conditions.
2. tool_path should contain a multi-tool chain arranged in the correct execution order, supporting cross-attribute analysis and constraint-based reasoning over multiple candidate objects.
3. The question must not directly mention any tool name, but should use standard scientific terminology to clearly describe the candidate objects, target properties, and scientific constraints; it must not describe the solving steps or reveal any tool outputs, numerical results, or answer hints.
4. The Parameter field should contain only the candidate substance names or IDs involved in the question, with a consistent representation format.
5. The question must not use comparison criteria that are unnatural in real scientific queries, such as median; instead, it should use scientific constraints such as maximum/minimum values, threshold-based judgments, or structural inclusion.
6. Ensure natural, diverse phrasing and avoid template-like wording.

Strictly output the response in **valid JSON** format only. The required **JSON** format is as follows:
{
"tool_path": ["tool_selected", "xxx"],
"question": "question text",
"Parameter": ["candidate substance 1", "candidate substance 2", "xxx"]
}

Level-4 (5–10 tools) consists of research strategy tasks, where the question simulates realistic scientific scenarios with open-ended research goals and multi-dimensional evaluation criteria. These tasks typically involve long tool chains and multi-objective trade-offs, requiring the model to autonomously plan a complete analysis workflow and formulate interpretable research decisions.

Prompt for Generating Level-4 Questions

System Prompt:
You are a science expert.

User Prompt:
Please generate one scientific tool-use question based on the given candidate tools. The question type should be one of Question & Answer, Multiple-Choice, Content Completion, and True/False.

Candidate Tools:
{tools_str}

Task Objective:
Generate a research strategy-level task. The question should simulate a realistic scientific scenario, specify an open-ended research objective and multi-dimensional evaluation criteria, and typically involve a long tool chain and multi-objective trade-offs, requiring the model to autonomously plan the analysis workflow and produce a rigorous scientific decision.

[RULE]:
1. The question must present a complete and realistic scientific motivation, avoiding a rote list of tool calls; it may focus on drug screening, materials selection, property prediction, mechanism investigation, safety assessment, or de novo design validation.
2. The question should use 1-2 sentences to establish the research background, followed by precise and non-redundant scientific statements specifying the analysis objective and evaluation criteria.
3. tool_path should contain a relatively long tool chain arranged in a reasonable execution order, supporting multidimensional analysis, evidence integration, and scientific decision-making.
4. The question must not directly mention any tool name, but should use standard scientific terminology to describe the research background, analysis objective, and evaluation criteria; it must not describe the specific solving steps or reveal any tool outputs, numerical results, or answer hints.
5. The Parameter field should contain the specific substances, sequences, or other scientific objects selected in the question.
6. Ensure natural, coherent, and situated in a realistic research context.

Strictly output the response in **valid JSON** format only. The required **JSON** format is as follows:

```
{
  "tool_path": ["tool_selected", "xxx"],
  "question": "question text",
  "Parameter": ["scientific object 1", "scientific object 2", "xxx"]
}
```

D Benchmark Validation

This section details the benchmark validation protocol of OpenSciToolBench, which consists of two stages, namely LLM-as-a-Judge verification and human validation (Yu et al. 2026).

LLM-as-a-Judge. We used Gemini-3.1-Pro as the judge to assess each generated sample along three dimensions: (1) whether the question is clearly stated and free of obvious ambiguity; (2) whether the reference tool chain fully covers the scientific operations required by the question; and (3) whether the input-output dependencies between adjacent tools in the chain are valid. Samples that failed this verification process were removed.

Prompt for Sample Quality Evaluation

System Prompt:
You are a scientific evaluation expert.

User Prompt:
Below are a question, a reference tool chain, and the corresponding parameter. Please use the following rule to determine whether the sample satisfies the quality standard.

[RULE]:
1. Check whether the question is clearly stated and free of obvious ambiguity.
2. Check whether tool_path covers the scientific operations required by the question.
3. Check whether the input-output dependencies between adjacent tools in tool_path are valid and can form an executable chain.
4. Judge as "No" if any condition fails; otherwise, "Yes".

Question:
{question}

Reference tool chain:
{tool_path}

Parameter:
{parameter}

Strictly output the response in **valid JSON** format only. The required **JSON** format is as follows:

```
{
  "decision": "Yes / No",
  "reason": "brief explanation"
}
```

Human Validation. To further ensure the quality and rigor of our dataset, we conducted human validation on the samples that passed the initial LLM-based verification. For Levels 1–3, we invited five students with backgrounds in biology, chemistry, and materials science to perform the review. For Level-4, we invited three domain experts, whose expertise was better suited for evaluating the open-ended research scenarios. Each sample was independently reviewed by three annotators according to the following four criteria:

- **Level Alignment**: whether the question effectively tests the design objective of the assigned level and accurately reflects the core competency intended to be evaluated at that level.
- **Semantic Clarity**: whether the question is clearly stated, logically coherent, and free of obvious ambiguity, such that the scientific goal and constraints can be accurately understood.
- **Tool-Chain Coherence**: whether the reference tool chain can fully support the question, and whether the input-output dependencies between consecutive tools are valid, avoiding mismatched tool capabilities or broken data flows.
- **Answerability**: whether the involved scientific entities exist, and whether the answer can be obtained through executing the given tool chain and applying the necessary model reasoning.

Student reviewers were compensated at a rate of $10 for every 20 cases, while expert reviewers received $15 for every 10 cases. The entire validation process took five days and cost $1,900 in total. Each reviewer independently provided a "Yes" or "No" judgment for every assigned sample. We then applied a majority-vote rule to aggregate the three reviewers' assessments and obtain the final consensus decision. Among the samples entering human validation, 85.7% satisfied the above criteria, resulting in a final benchmark of 900 questions across four difficulty levels, three scientific domains, and four question formats. Detailed dataset statistics are reported in Tables A5.

E Ontology-Aware Inference Algorithm

Algorithm 1 presents the details of ontology-aware inference, including skill and task routing, active tool acquisition, and knowledge update.

F Active Tool Request Analysis

In this section, we provide a brief analysis of why SciToolAgent-Evo adopts active tool requests instead of passive tool selection.

Let $\bar{\mathcal{T}} = \mathcal{T}_K \cup \tilde{\mathcal{T}}_U$ denote the conditionally observable tool space, where $\mathcal{T}_K$ is the known tool graph and $\tilde{\mathcal{T}}_U$ contains

Level	#Questions	#Train	#Test	Q&A	MCQ	CC	T/F	Avg. Path	Max. Path	Avg. #Param.	Avg. #Cons.	Avg. #Attr.	Avg. Depth	#Reuse
Level-1	262	157	105	253	0	4	5	1.57	2	1.00	0.00	1.14	1.00	54
Level-2	238	143	95	63	79	31	65	2.26	4	3.23	1.84	1.45	2.18	31
Level-3	213	128	85	39	70	63	41	3.68	6	4.11	3.25	2.61	3.52	26
Level-4	187	111	76	89	35	28	35	5.21	10	1.61	4.76	4.12	4.88	17
Total	900	539	361	444	184	126	146	3.03	10	2.45	2.24	2.19	2.71	128

Table A5: Statistics of the **OpenSciToolBench** dataset across four difficulty levels, covering dataset scale, question formats, tool-path structure, and task complexity. *Avg. Path* and *Max. Path* denote the average and maximum tool-path length; *Avg. #Param.*, *Avg. #Cons.*, and *Avg. #Attr.* denote the average number of input parameters, constraints, and tool attributes per question; *Avg. Depth* denotes the average reasoning depth, and *#Reuse* denotes the number of reusable skills.

Algorithm 1: Ontology-Aware Inference

Require: Test task q, task domain d_q, memory $\mathcal{M} = (G_K, \mathcal{K}, \mathcal{E})$, unknown tools T_U, bandit gate $\mathcal{B}$
Ensure: Answer $\hat{y}$, updated $\mathcal{M}$, T_U and $\mathcal{B}$

$\hat{\tau} \leftarrow [\,], \mathcal{A} \leftarrow [\,]$
Step 1: Skill and task routing
$k^\star \leftarrow \textsc{SkillMatch}(q, \mathcal{K})$
if $k^\star \neq \emptyset$ **then**
  $\hat{\tau} \leftarrow$ Tool chain stored in $k^\star$
  $\hat{y} \leftarrow \textsc{Execute}(q, \hat{\tau})$
  return $\hat{y}, \mathcal{M}, T_U, \mathcal{B}$
end if
$c_q \leftarrow \textsc{AssessDifficulty}(q)$
if $c_q = \textsc{Simple}$ **then**
  $\hat{\tau} \leftarrow$ CoT planning over $G_K \cup T_U$
  $\hat{y} \leftarrow \textsc{Execute}(q, \hat{\tau})$
  return $\hat{y}, \mathcal{M}, T_U, \mathcal{B}$
end if
Step 2: Active Tool acquisition
$\mathcal{E}_q \leftarrow \mathcal{E}_{gen} \cup \textsc{TopK}(\mathcal{E}_{d_q}, q)$
$R_q = \{r_1, \ldots, r_l\} \leftarrow \textsc{ActiveRequest}(q, \mathcal{E}_q)$
for each request $r_j \in R_q$ **do**
  $C_j^0 \leftarrow \textsc{TopK}_{t \in V_K} S(r_j, t)$
  $C_j \leftarrow \textsc{SubgraphExpand}(C_j^0, G_K)$
  $x_j = [f_j, m_j, s_j, p_j]^\top \leftarrow \textsc{GateCtx}(r_j, C_j)$
  $a_j \leftarrow \mathcal{B}(x_j)$, where $a_j \in \{\textsc{Known}, \textsc{Unknown}\}$
  if $a_j = \textsc{Known}$ **then**
    $t_j \leftarrow \textsc{Top}(C_j)$
  else
    $t_j \leftarrow \textsc{SearchUnknown}(r_j, T_U)$
  end if
  $\hat{\tau} \leftarrow \hat{\tau} \oplus t_j, \quad \mathcal{A} \leftarrow \mathcal{A} \oplus (x_j, a_j)$
end for
Step 3: Knowledge update
$(\hat{y}, H, r) \leftarrow \textsc{Execute}(q, \hat{\tau})$, where $r \in \{0, 1\}$
if $r = 1$ **then**
  for each unknown tool $u \in \hat{\tau}$ **do**
    $o_u \leftarrow \textsc{CompleteOntology}(u, H)$
    $G_K \leftarrow \textsc{Migrate}(G_K, u, o_u)$
    $T_U \leftarrow T_U \setminus \{u\}$
  end for
  $\mathcal{K} \leftarrow \mathcal{K} \cup \textsc{SkillDistill}(q, \hat{\tau}, H)$
end if
$\mathcal{M} \leftarrow (G_K, \mathcal{K}, \mathcal{E})$
return $\hat{y}, \mathcal{M}, T_U, \mathcal{B}$

unknown-tool attributes accessed only when G_K is insufficient. Let $\Pi(\bar{\mathcal{T}})$ denote the candidate tool-chain space and τ^* the reference tool chain for task q.

A passive strategy directly matches the raw task against candidate tool chains:

$$P_{\text{passive}}(\tau^* \mid q, \bar{\mathcal{T}}) = \frac{P(q \mid \tau^*, \bar{\mathcal{T}}) P(\tau^* \mid \bar{\mathcal{T}})}{\sum_{\tau \in \Pi(\bar{\mathcal{T}})} P(q \mid \tau, \bar{\mathcal{T}}) P(\tau \mid \bar{\mathcal{T}})}. \quad (7)$$

This direct query-based matching is inefficient in large heterogeneous tool spaces and unreliable for weakly described unknown tools.

In contrast, given q and retrieved experiences $\mathcal{E}_q$, SciToolAgent-Evo actively generates a request set $R_q = \{r_1, r_2, \ldots, r_l\}$, where each r_j is a concise description of one tool need. Tool-chain acquisition is then conditioned on these requests:

$$P_{\text{active}}(\tau^* \mid q, \mathcal{E}_q, \bar{\mathcal{T}}) = \sum_{R_q} P(\tau^* \mid R_q, \bar{\mathcal{T}}) P(R_q \mid q, \mathcal{E}_q). \quad (8)$$

Here, $P(R_q \mid q, \mathcal{E}_q)$ captures the agent's ability to express task-specific tool needs under ontology-guided reasoning.

From an information-acquisition view, an effective request set should reduce uncertainty over the required tool chain:

$$\begin{aligned} R_q^* &= \arg\max_{R_q} I(\tau^*; R_q \mid q, \mathcal{E}_q) \\ &= \arg\max_{R_q} \left[H(\tau^* \mid q, \mathcal{E}_q) - H(\tau^* \mid R_q, q, \mathcal{E}_q) \right]. \end{aligned} \quad (9)$$

Moreover, functionality-level requests are better aligned with both known-tool ontology and unknown-tool functionalities than raw task queries, since they describe the required capability in the same semantic form as tool descriptions. Thus, active tool requests transform scientific tool acquisition from costly raw-query matching into requirement-conditioned retrieval.

G Experimental Settings

During knowledge accumulation, we first sample a deterministic baseline trajectory ($T_{\text{base}} = 0$). If this trajectory fails, we further sample three exploratory trajectories in parallel with $T_{\text{exp}} \in \{0.2, 0.5, 0.8\}$. After accumulation, we deduplicate the experience pool by removing experience rules whose pairwise semantic similarity exceeds $\theta_{\text{sim}} = 0.85$.

At inference time, the agent utilizes all global experience rules and retrieves the top-5 domain-specific entries. The decoding temperature of the Main Agent is set to 0.3, with max retries = 2, and the final answer is evaluated by GPT-5-mini. We split the toolset into two equal partitions, designating 50% as known and the remaining 50% as unknown. The difficulty routing module uses two branch thresholds, namely an estimated tool count of 2 and a scientific entity count of 4. During this stage, locally deployed Qwen3-Embedding-0.6B is used as the retrieval model ($max_tools = 5$), while Qwen3-1.7B is used as the reranking model. We implement a LinUCB-based bandit gate with a 4D context vector. The exploration coefficient α initializes at 0.8 and decays by 0.01 per step to a minimum of 0.2. Furthermore, a 20-task sliding window monitors the known-tool failure rate, dynamically rescaling α within $[0.7, 1.3]$ to adaptively balance exploration and exploitation.

Specifically, we adopt a unified and standardized downstream pipeline for tool execution, summarization, and evaluation. This setup ensures a fair comparison by confining methodological variations strictly to the tool-chain planning strategies.

H Introduction of Baselines

We have selected the following methods as our baselines:

- **Direct**: Generates the final result directly without intermediate reasoning trajectories. In our setting, the model first identifies suitable tools from the known set, retrieves candidates from the unknown set if necessary, and directly outputs the planned tool chain for execution and summarization.
- **CoT** (Wei et al. 2022): Produces an explicit reasoning process prior to generating the result. Specifically, it reasons over the known and unknown tool sets sequentially to identify required tools, subsequently merging them into a coherent chain for execution.
- **RAG** (Lewis et al. 2020): Augments tool-chain planning with retrieved tool candidates. It retrieves relevant tools from the known graph and, when capability gaps are identified, retrieves additional candidates from the unknown set before constructing the tool chain for execution.
- **ReAct** (Yao et al. 2023): Integrates reasoning with action execution through an iterative Thought-Action-Observation loop. The model generates a reasoning trajectory to select and execute actions, dynamically updating its plan and exploring unknown tools based on real-time observations until completion.
- **Reflexion** (Shinn et al. 2023): Improves agent planning through verbal self-reflection. When an attempt fails, the model analyzes the failure mode to generate feedback, which then informs and refines subsequent planning attempts.
- **SciToolAgent** (Ding et al. 2025): Leverages a scientific tool knowledge graph to facilitate multi-tool orchestration. It retrieves relevant tools from the graph and searches the unknown set when gaps are identified, following which it synthesizes and executes the tool chain.

I Detailed Prompts for Agent Workflow

To ensure the robustness and reproducibility of SciToolAgent-Evo, we present the core prompt templates used in our workflow, covering active tool requirement description, candidate tool reranking and scoring, experience rule generation, and reusable skill extraction.

Prompt for Active Tool Description

System Prompt:
You are a science expert.

User Prompt:
You need to describe the tool capabilities required for a scientific task.

Experience: {experience_section}

Task:
{question}

Previous Error (if any):
{error_text}

[IMPORTANT RULE]:
1. Each tool capability must be an atomic computational primitive, used only to derive one property or perform one transformation. It must not merge multiple properties or operations, nor include decision logic such as comparison, filtering, or aggregation.
2. Use the fewest and most direct steps for task analysis. If a single tool capability can directly produce the target result, do not introduce redundant preprocessing, intermediate transformations, or repeated computations. Each type of property should be computed only once.
3. If the task involves state or representation transformation, tool capabilities must follow data-form evolution, ensuring causal coherence between steps without skipping intermediate forms.
4. Tool descriptions should be generic capability descriptions and must not include specific substance names, numerical values, candidate answers, task conclusions, or instantiated information.
5. If the task involves multiple substances, sequences, or scientific objects, describe only the tool capabilities required for processing one object; do not introduce cross-object comparison or selection logic.
6. Use goal-oriented backward reasoning by first determining the final output internally, then inferring the required inputs and tool capabilities, and finally outputting only the tool capability descriptions.
7. Each tool description must be highly concise, with no more than 15 English words.

Now analyze the given task and list all required tool capabilities.

The required output format is as follows:
(1) tool description 1
(2) tool description 2
...

Do not output any additional explanation.

Prompt for Rerank and Score

System Prompt:
You are a science expert.

User Prompt:
Complete the following two-step task.

Task Requirement:
{tool_request}

Task Constraints:
Category={target_category}{expected_input}

Candidate Tools:
{candidates_text}

Step 1:
Analyze each candidate tool based on its Function description, Input type, Output type, and Category. Select the best-matching tool as top-1 and the second-best tool as top-2.

Step 2:
Score the selected top-1 and top-2 tools according to the following criteria:

1. known_top1_feasibility $(0.0–1.0)$: the overall feasibility of top-1.

- 1.0: Function fully matches, with aligned I/O and Category.
- 0.75: Function strongly matches, with minor I/O or Category mismatch.
- 0.5: Function partially matches, with moderate I/O or Category mismatch.
- 0.25: Function is weakly relevant, with clear I/O or Category mismatch.
- 0.0: Function is completely mismatched.

2. known_margin: the rationale for prioritizing top-1 over top-2.

- 1.0: top-1 is clearly better than top-2.
- 0.75: top-1 is slightly better than top-2.
- 0.5: top-1 and top-2 are comparable.
- 0.25: top-2 is slightly better than top-1.
- 0.0: top-2 is clearly better than top-1.

3. ontology_alignment_score $(0.0–1.0)$: the overall ontology-level alignment between top-1 and the task.

- 1.0: Category and I/O types are fully aligned.
- 0.75: Category matches, with minor I/O type mismatch.
- 0.5: Category matches but I/O types differ, or I/O matches but Category differs.
- 0.25: Category and I/O both differ but remain partially relevant.
- 0.0: Category or I/O types are completely mismatched.

Strictly output the response in **valid JSON** format only. The required **JSON** format is as follows:

```
{
  "top1": "ToolName1",
  "top2": "ToolName2",
  "known_top1_feasibility": F,
  "known_margin": M,
  "ontology_alignment_score": O
}
```

Prompt for Generating Experience Rules

System Prompt:
You are a science expert.

User Prompt:
Based on the diagnosis and the rules, generate highly reusable experience rules.

Question:
{question}

Diagnosis:
{diagnosis}

Please generate the following fields:

1. general_prompt: One general principle applicable to scientific tool planning.
2. domain: Strictly choose one from the following: biology, chemistry, materials.
3. domain_prompt: A robust tool mapping rule targeting the diagnosed error.

The core expression must follow:
"When requiring [specific scientific context/constraint], select a tool that provides [specific functional capability]."

[RULE]:

1. All rules must provide clear and actionable operational guidance.
2. Limit each rule to 25 words.

Strictly output the response in **valid JSON** format only. The required **JSON** format is as follows:
{
"general_prompt": "1 universal actionable rule",
"domain": "biology/chemistry/materials",
"domain_prompt": "domain-specific rule"
}

Prompt for Generating Reusable Skills

System Prompt:
You are a science expert.

User Prompt:
Given a specific task and its successfully executed tool chain, extract one standardized and reusable skill.

Task:
{question}

Tool Chain:
{tool_path}

Core Extraction Criteria:
1. Avoid over-abstraction: preserve specific scientific properties.
2. Remove concrete parameters: do not include numerical values, thresholds, filtering conditions, or specific substance names.
3. Skill naming: the key Ni should use short snake_case and reflect only the property/function combination.
4. Skill description: the key Zi must be a coherent sequence of atomic computational primitives. Use connectives to form one fluent description within 20 words.

Strictly output the response in **valid JSON** format only. The required **JSON** format is as follows:
{
"Ni": "property_combination_name",
"Zi": "A cohesive English sentence describing the property extraction pipeline."
}

J Dataset Examples

In this section, we provide representative questions from OpenSciToolBench, illustrating one example for each of the four difficulty levels.

Level-1

Question:
For the molecule CCCCCCCCCCCCCCCCC-C/C=C\CCCCCCCC(=O)O, what is its molecular weight, and what is its Kappa1 topological descriptor?

Parameters:
CCCCCCCCCCCCCCCCCC/C=C\CCCCC
CCC(=O)O

Standard_Toolpath:
["SMILESToWeight", "GetKappa1"]

Level-2

Question:
First convert the molecule names: Propionamide, Methane, Ethane, and Butane, to their respective SMILES representations. True or False: among these four molecules, the compound that contains at least one amide bond also has a larger number of heavy atoms than any of the other three.

Parameters:
Propionamide, Methane, Ethane, Butane

Standard_Toolpath:
["NameToSMILES", "GetAmideBondsNum", "GetHeavyAtomsNum"]

Level-3

Question:
First convert the following SLN strings to SMILES: B-H-B3, CH3CH2C[*]HCH2Br, O[1]CH2CH(@1)CH3, C[1][H2]CH2CH2
CH2CH2@1, CH3CH3, and C[1]:C:CH:
CH:CH:CH:@1. Among the converted molecules, identify the one that simultaneously satisfies all of these conditions: (1) it is achiral, meaning its stereo code indicates no chirality; (2) it contains no ring system; (3) its Chi3v value is nonzero, indicating an acyclic branching pattern; and (4) it has the highest asphericity among those meeting conditions (2)–(4), indicating it is the least spherical.

Options:
A. B-H-B3

B. CH3CH2C[*]HCH2Br
C. O[1]CH2CH(@1)CH3
D. C[1][H2]CH2CH2CH2CH2@1
E. CH3CH3
F. C[1]:C:CH:CH:CH:CH:@1

Parameters:
B-H-B3, CH3CH2C[*]HCH2Br, O[1]CH2CH(@1)CH3, C[1][H2]CH2CH2CH2CH2@1, CH3CH3, C[1]:C:CH:CH:CH:CH:@1

Standard_Toolpath:
["SlnToSmiles", "GetStereoCodeFromSmiles", "GetRingsNum", "GetChi3v", "GetAsphericity"]

Level-4

Question:
A synthetic biology team is developing a compact DNA chassis for stable mammalian expression and wants to understand whether a newly generated sequence is a viable regulatory element rather than a problematic genomic insert. The sequence should be examined from sequence composition through structural plausibility and expression-related properties so that the team can decide whether it is worth advancing to experimental validation.
Generate one random DNA sequence. Predict whether it contains CpG islands that could trigger epigenetic silencing, and identify restriction enzyme recognition sites that could compromise cloning stability. Calculate its molecular weight in both double-stranded linear and single-stranded circular forms for construct planning. Identify the longest open reading frame, translate it into an amino-acid sequence, convert the resulting protein into a 3D structure, compute its hydrophobicity profile, and predict its overall solubility. Based on this integrated characterization, is the generated sequence more suitable as a benign engineered element or as a candidate likely to cause manufacturing or expression problems, and which features most strongly support that conclusion?

Parameters:
None

Standard_Toolpath:
["RandomDNAGeneration", "CpGIslandPrediction", "SummaryEnzymeCleavageSites", "CalculateDoubleLinearDNAMolecularWeight", "CalculateSingleCircleDNAMolecularWeight", "FindORF", "TranslateDNAtoAminoAcidSequence", "SequenceToPdb", "ComputeProtScale", "ProteinSolubilityPredictor"]